

- *Multi-sink output*—Similar to dynamic resource allocation, but rather than producing a single output file, produce an output file for each input file in `fahrenheits.txt`. Name the output file by appending `.celsius` onto the input file name.

Can these capabilities be incorporated into our definition of `Process` in a way that preserves resource safety? Yes, they can! We actually already have the power to do these things, using the `flatMap` combinator that we’ve already defined for an arbitrary `Process` type.

For instance, `flatMap` plus our existing combinators let us write this first scenario as follows:

```
val convertAll: Process[IO,Unit] = (for {
  out <- fileW("celsius.txt").once
  file <- lines("fahrenheits.txt")
  _ <- lines(file).
    map(line => fahrenheitToCelsius(line.toDouble)).
    flatMap(celsius => out(celsius.toString)).
} yield ()) drain
```

Trim the stream to, at most, a single element; see chapter code.

This code is completely resource-safe—all file handles will be closed automatically by the runner as soon as they’re finished, even in the presence of exceptions. Any exceptions encountered will be thrown to the `runLog` function when invoked.

We can write to multiple files just by switching the order of the calls to `flatMap`:

```
val convertMultisink: Process[IO,Unit] = (for {
  file <- lines("fahrenheits.txt")
  _ <- lines(file).
    map(line => fahrenheitToCelsius(line.toDouble)).
    map(_ toString).
    to(fileW(file + ".celsius"))
} yield ()) drain
```

And of course, we can attach transformations, mapping, filtering, and so on at any point in the process:

```
val convertMultisink2: Process[IO,Unit] = (for {
  file <- lines("fahrenheits.txt")
  _ <- lines(file).
    filter(!_ startsWith("#")).
    map(line => fahrenheitToCelsius(line.toDouble)).
    filter(_ > 0). // ignore below zero temperatures
    map(_ toString).
    to(fileW(file + ".celsius"))
} yield ()) drain
```

There are additional examples using this library in the chapter code.

15.4 Applications

The ideas presented in this chapter are widely applicable. A surprising number of programs can be cast in terms of stream processing—once you’re aware of the abstraction, you begin seeing it everywhere. Let’s look at some domains where it’s applicable: